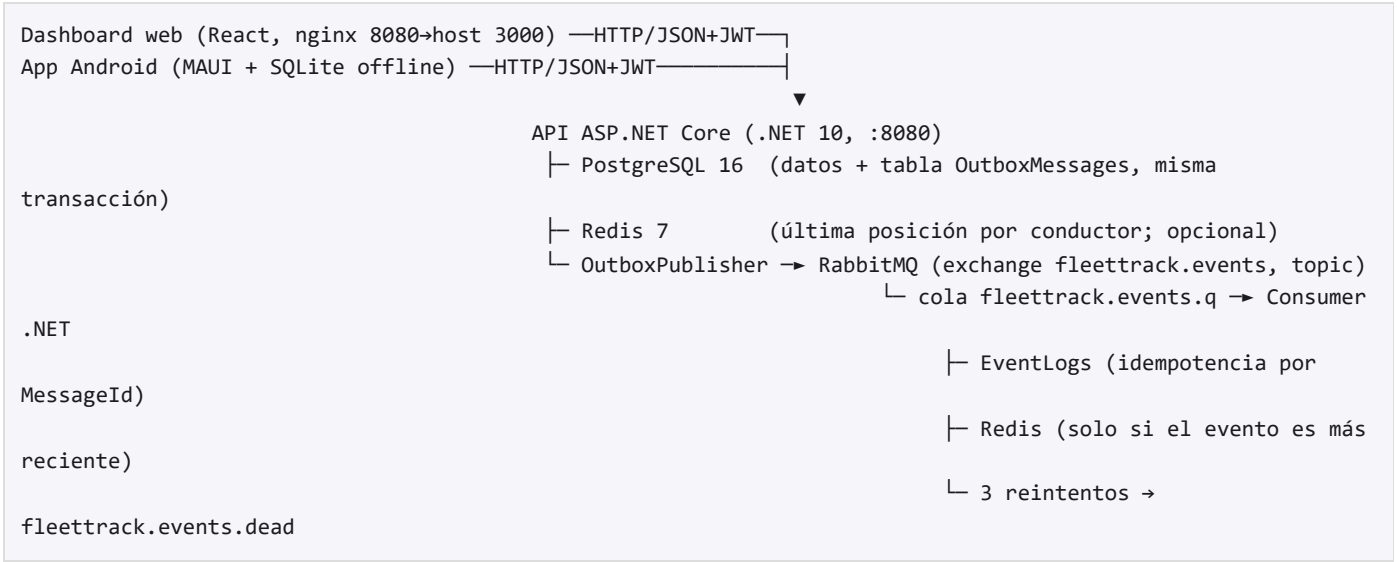
FleetTrack — Arquitectura

Proyecto final, curso 048 Aseguramiento de la Calidad del Software (Universidad Mariano Gálvez).
Documento consolidado por Alejandro.

1. Propósito y alcance

FleetTrack gestiona rutas, entregas y seguimiento de conductores. El **supervisor** administra clientes, vehículos, conductores y rutas desde un dashboard web y ve la última posición de cada conductor. El **conductor** usa una app Android: consulta su ruta, inicia/finaliza/falla entregas, adjunta una fotografía como evidencia y envía su ubicación; todo funciona sin conexión y se sincroniza al reconectar.

2. Vista general



3. Componentes

Componente	Tecnología	Responsabilidad
`frontend/`	React 19, TypeScript, Vite; servido por nginx sin privilegios	Login de supervisor, CRUD de clientes/vehículos/conductores, asignación de rutas, dashboard con refresco cada 5 s y aviso de modo degradado
`backend/src/FleetTrack.Api`	ASP.NET Core, EF Core, Npgsql	API REST, autenticación JWT, reglas de negocio, outbox, caché de posición
`backend/src/FleetTrack.Domain`	.NET	Entidades, enums, eventos de dominio y topología de mensajería

Componente	Tecnología	Responsabilidad
`backend/src/FleetTrack.Consumer`	.NET Worker	Consume eventos de forma idempotente, actualiza caché, reintentos y cola de fallidos
`mobile/`	.NET MAUI (solo Android), SQLite	App del conductor, cola de operaciones pendientes y `SyncService`
`infrastructure/terraform`	Terraform (Azure)	VM de QA, red y reglas; identidad administrada
`ansible/`	Ansible	Docker, despliegue del stack y nginx como reverse proxy en la VM
`Jenkinsfile`	Jenkins declarativo	Pipeline de calidad y entrega controlada

4. Modelo de datos

Entidades principales: `AppUser` (rol Supervisor o Driver), `Driver`, `Vehicle` (placa única), `Customer`, `DeliveryRoute` (estado Planned → InProgress → Completed), `Delivery` (Pending → InProgress → Completed o Failed, con orden dentro de la ruta), `DeliveryEvidence` (foto, tipo, tamaño), `TrackingEvent` (latitud, longitud, `EventId` único, `RecordedAt`), `OutboxMessage` y `EventLog` (clave = `MessageId`).

5. Decisiones de diseño relevantes

Decisión	Motivo
Outbox transaccional	El cambio de negocio y el evento se guardan juntos; si RabbitMQ cae la API sigue respondiendo y el evento se publica después (entrega al-menos-una-vez, con confirmación del broker).
Idempotencia en tres niveles	API: `start/complete/fail` bajo `SELECT … FOR UPDATE` sobre la ruta (un solo efecto con peticiones concurrentes); tracking por `eventId`; consumidor por `MessageId`.
Eventos fuera de orden	La caché de última posición nunca se reemplaza por un `RecordedAt` anterior; un estado final no retrocede (completar una entrega fallida → 409).
Degradación sin Redis	Redis es una caché. Sin él, `GET /api/tracking/latest` lee PostgreSQL y responde `X-Redis-Status: unavailable`; el dashboard avisa.
Cola offline en móvil	Toda operación se guarda en SQLite y se reenvía en orden; se detiene ante un fallo de red y descarta lo que el servidor rechaza con 4xx definitivo (salvo 401/408/429).
Contenedores sin privilegios	API y consumer con `USER $APP_UID`; web con `nginx-unprivileged`.

6. Seguridad

- Autenticación JWT; dos roles. El conductor solo opera sus propias entregas (la API verifica la pertenencia) y el supervisor accede a la administración.
- Fotografías: validación de tipo y tamaño, nombre generado por el sistema, acceso autenticado.

- Cabeceras en nginx (CSP, anti-clickjacking, nosniff, Permissions-Policy, sin versión de servidor) y en la API (`nosniff`, `Cross-Origin-Resource-Policy: same-origin`).
- Android: HTTP sin cifrar permitido solo hacia `10.0.2.2` y `localhost` (`network_security_config`), para el emulador de QA.
- Secretos por variables de entorno o credenciales de Jenkins; `.env` fuera de Git. Credenciales del seed son de prueba.

7. Ambientes y despliegue

Ambiente	Cómo se levanta	Uso
Local / QA	`docker compose up -d --build` (postgres en 5433, redis, rabbitmq, api 8080, consumer, web 3000) con healthchecks y reinicio automático	Desarrollo y todas las pruebas
Android	Emulador API 35 (AVD `fleettrack`), API en `10.0.2.2:8080`	Maestro y validación manual
Azure QA	Terraform (VM + red) y Ansible (Docker, stack, nginx)	Código listo; sin ejecutar por no tener la suscripción confirmada
CI/CD	`Jenkinsfile`: agente `qa` y agente `android`; SonarQube, Dependency-Check y emulador en equipos propios	Ver plan de pruebas, sección 5

8. Calidad y observabilidad

`/health` en la API y healthchecks de Compose en todos los servicios. Reportes por herramienta en `reports/` (Sonar, Semgrep, ZAP, k6, cobertura) con fecha, commit y ambiente en `reports/metadata.json`. La consola de RabbitMQ (puerto 15672) muestra colas, conexiones y la cola de fallidos.

9. Limitaciones conocidas de la arquitectura

- La sesión del móvil vive en memoria: tras reiniciar la app hay que volver a iniciar sesión (las operaciones pendientes se conservan).
- Los reintentos del consumidor esperan en línea; suficiente para la carga probada, a cambiar por colas de retraso si crece.
- El outbox no se purga y los mensajes fallidos se reprocesan a mano desde RabbitMQ.
- Un solo nodo por servicio: sin alta disponibilidad.